

Cache Lab: Implement Cache-Aside Read Pattern

 systemdesignschool.io/fundamentals/cache-lab-read



Lab: The Read Drill

The previous article explained cache-aside and read-through as concepts. This lab implements cache-aside from scratch so you can see the hit/miss mechanics in action.

The Exercise

Implement a `get_user()` function that follows the cache-aside pattern:

1. Check the cache for the key
2. On hit: return cached data (fast path)
3. On miss: query the "database," store in cache, return (slow path)

The code below simulates a database with artificial latency. Run it to observe how repeated reads shift from slow (miss) to fast (hit).

Cache-Aside: `get_user()` Implementation

```
# Simulated database (slow — each read costs 50ms)
1: {"name": "Alice", "email": "alice@example.com", "role": "admin"},
2: {"name": "Bob", "email": "bob@example.com", "role": "user"},
3: {"name": "Charlie", "email": "charlie@example.com", "role": "user"},
"""Simulate a slow database read."""
time.sleep(0.05) # 50ms latency
return DATABASE.get(user_id)
```

```
# The cache (a simple dictionary)
"""Cache-aside: check cache first, fall back to DB on miss."""
key = f"user:v1:profile:{user_id}"
# Step 1: Check cache
return cache[key], "HIT"
# Step 2: Cache miss — query database
data = db_query(user_id)
return None, "MISS (not found)"
# Step 3: Store in cache for next time
return data, "MISS (stored)"
# --- Run the experiment ---
print("=== First Round: All Misses (cold cache) ===")
```

Observe how the first round takes ~50ms per unique user (database read), while the second round completes nearly instantly (memory lookup). The first three calls in round one are misses. Calls 4-6 in round one are hits because users 1, 2, 3 were already cached.

Try It: Cache Invalidation

What happens when data changes? Extend the lab by adding a `delete_from_cache()` function that removes a key, forcing the next read to refetch from the database.

Cache-Aside: Invalidation

This demonstrates the full cache-aside lifecycle: read (miss) → cache → read (hit) → invalidate → read (miss again with fresh data). The write-pattern section explores strategies for keeping the cache consistent when data changes, without requiring manual invalidation at every write.